The EPIC Framework for Spec-Driven Development

BRUNO CLAUDINO*, Virginia Commonwealth University, USA
RUIZHE XING, Oregon State University, USA
YUANHAO ZUO, Virginia Commonwealth University, USA
TYLER MENEZES, CodeDay, USA
ANITA SARMA, Oregon State University, USA
KOSTADIN DAMEVSKI, Virginia Commonwealth University, USA
ZIXUAN FENG, Virginia Commonwealth University, USA

arXiv:2610.07534v1 [cs.SE] 6 Oct 2026

One practitioner we interviewed said their team writes "must" instead of "should" when instructing a coding agent, because the agent may treat "should" as optional. Small wording choices matter because agents often fill gaps in their instructions with their own assumptions. Spec-driven development (SDD) asks developers to write a specification, plan, and tasks before the agent writes code. SDD frameworks provide templates for these artifacts, but the templates do not help developers judge whether they have written enough or clearly enough. We studied what good SDD specifications contain. We scored the artifacts of 114 open-source SDD repositories against ISO/IEC/IEEE 29148 and derived practices from the highest-scoring ones. The resulting framework, **EPIC**, has 40 practices in 10 quality dimensions that guide developers in making expectations and decisions explicit in specifications, plans, and tasks for coding agents. The majority of SDD practitioners (N=15) endorsed every practice. Repositories in the top third of specification quality spent 11.8% of their commits on bug fixes, compared with 20.4% in the bottom third, and had a median of 4x as many contributors. Developers can use **EPIC** to find and fill the gaps in a specification before the agent does.

CCS Concepts: • **Human-centered computing**;

1 Introduction

Ask a colleague to make a query "fast," and they will likely ask how fast, under what workload, and how the result will be checked. Ask an AI coding agent, and it may fill the gap with its own assumptions and return code that appears to answer the request [61]. Agents struggle to recognize when information is missing, even when asking would improve their solutions [79]. As one practitioner we interviewed put it, the agent is "*making a decision even if I don't*" (P10). Once the code exists, it is difficult to tell which assumptions came from the developer and which came from the agent.

The underlying problem is not new. Software has always been only as good as the account of what it should do, and requirements engineering has long held that this account must state not only the required behavior but also the assumptions, constraints, and operating conditions under which the software must work [45, 82]. Incomplete and inconsistent requirements remain common problems in industry, associated with rework and overruns [56]. Coding agents raise the stakes. Producing code faster does not necessarily mean producing better software: in a study of Cursor adoption, gains in development velocity were temporary, while increases in static-analysis warnings and code complexity persisted [43]. As developers increasingly orchestrate agents rather than write every line themselves, the specification becomes the main place where they decide what gets built, and the one artifact they can review before any code exists.

Spec-driven development (SDD) puts this specification at the center of the workflow: a structured natural-language specification drives the agent's implementation [3]. The practice barely existed

Authors' Contact Information: Bruno Claudino, claudinomab@vcu.edu, Virginia Commonwealth University, USA; Ruizhe Xing, Oregon State University, USA, xingru@oregonstate.edu; Yuanhao Zuo, Virginia Commonwealth University, USA, zuoy@vcu.edu; Tyler Menezes, CodeDay, USA, tylermenezes@codeday.org; Anita Sarma, Oregon State University, USA, anita.sarma@oregonstate.edu; Kostadin Damevski, Virginia Commonwealth University, USA, kdamevski@vcu.edu; Zixuan Feng, Virginia Commonwealth University, USA, fengz3@vcu.edu.

before 2025 and now spans more than 70,000 public GitHub repositories [3]. Yet developers have little basis for judging whether a specification says enough. SDD frameworks such as GitHub Spec Kit, Kiro, and OpenSpec provide templates and workflow steps [32, 41, 52], but a template shows where information belongs, not what is sufficient, and each framework structures it differently. A specification can fill every section and still leave assumptions unstated or contradict its plan. Requirements-quality standards define good requirements, individually and as a set [1, 28], but not the plans and tasks that SDD adds and that must agree with them. CAFE(S) defines qualities of the context an agent receives as a shared language rather than a measurement system [47]. Developers therefore lack concrete, empirically grounded guidance on what to articulate in SDD artifacts, and little evidence links requirements quality to development outcomes [34].

To our knowledge, this is the first study to derive SDD specification practices from both requirements-engineering standards and the artifacts developers write. We investigate two research questions:

- **RQ1:** What specification practices are used in SDD, and how do they address specification quality?
- **RQ2:** How does specification quality relate to software outcomes, and how do practitioners assess the clarity and usefulness of the identified practices?

Through a mixed-methods study, we established a quality lens from ISO/IEC/IEEE 29148:2018 and SDD framework guidance, applied it to artifacts from 114 open-source SDD repositories, derived practices from exemplar artifacts, and refined them using 30 grey-literature sources. The result, **EPIC**, organizes 40 practices for writing and reviewing SDD artifacts into ten quality dimensions and four categories: **E**xpressive, **P**recise, **I**nterpretable, and **C**ompartmentalized. Our study yields three main findings:

(1) **SDD shifts the unit of quality from the single requirement to the artifact set.** The three new dimensions **EPIC** adds to the seven grounded in ISO/IEC/IEEE 29148:2018 concern a specification, a plan, and tasks that must agree with one another. They cover the dependencies and constraints requirements rely on, separating required behavior from implementation decisions, and recording planning information.

(2) **Practitioners agree on the practices and on when they pay off.** None of the 30 grey-literature sources added a practice, and a majority of the 15 practitioners we interviewed endorsed every one of the 40. Their explanations show when a practice matters, so teams can adopt practices as a project grows rather than all at once.

(3) **Specification quality tracks with how repositories evolve.** Repositories in the top third of scores spent 11.8% of their commits on bug fixes versus 20.4% in the bottom third, had 15% lower median code complexity, and had a median of 12.5 contributors versus 3. We treat these observational associations as hypothesis-generating; bug and vulnerability density results were mixed.

In summary, this paper contributes: (1) **EPIC**, an empirically grounded framework of 40 specification practices for SDD artifacts; (2) practitioner accounts of when these practices are worth their cost; and (3) exploratory evidence that specification quality is associated with code quality and development activity. These contributions give developers concrete criteria for reviewing a specification before any code exists, an agent skill that applies them, and a structure for benchmarking how agents act on specifications.

2 Background

Software engineering has spent more than half a century trying to write down what a program should do before writing the program [45, 46, 82].

Fifty Years of Trying to Capture Intent. Early work on software specification and requirements engineering emphasized making intended behavior, interfaces, and assumptions explicit, helping engineers understand what to build and assess whether an implementation meets those expectations [42, 45, 46, 57, 82]. These approaches address a shared problem: when intent and implementation are separated, they can drift apart. Industrial surveys report incomplete and inconsistent requirements as common problems leading to rework and budget or schedule overruns [56, 80]. Approaches such as Test-Driven Development and Behavior-Driven Development were later developed to make intent more visible and easier to check [11, 64]. These approaches developed primarily around the assumption that a human would write the code.

From Vibe Coding to Specification-Driven Development. AI code generation allows natural-language instructions to guide implementation [18]. In *vibe coding*, developers describe desired outcomes and refine prompts after running the generated code [19, 27]. As conversations grow, requirements can become implicit or scattered across prompts, making completeness and consistency difficult to assess [10, 49, 76, 81]. Developers often struggle to understand, debug, and steer AI-generated code [54, 78]. SWE-chat documents user corrections in developer–agent interactions [10], while repository research shows agents overlooking documented contribution rules [81]. These findings highlight that the difficulty is not simply whether the model can write code. It is whether the developer can make their intent clear enough for the model to act on it.

SDD has regained attention among researchers and practitioners working on AI-assisted software development [23, 30, 68]. In SDD, developers express the desired outcome in reviewable natural language that guides planning, implementation, and validation across sessions [3, 23]. These specifications serve as a shared reference point between humans and agents [23, 68, 75]. In practice, SDD takes the form of lightweight document pipelines. For example, Kiro separates requirements, design, and tasks; GitHub Spec Kit follows `Specify` → `Plan` → `Tasks` → `Implement`; and OpenSpec organizes proposals, specifications, designs, tasks, and archives [32, 41, 52]. Despite their differences, these systems share a premise: natural-language intent should become a persistent artifact that mediates collaboration between humans and agents.

The Unresolved Problem of Specification Quality. A compiler may reject invalid input, and a colleague may ask for clarification. An agent may silently resolve ambiguity and return code that appears to meet the request [61, 79]. Small differences in how a requirement is phrased can substantially alter model outputs [59]; issuing the same prompt twice can produce materially different implementations [65]; and underspecified requirements are a measurable source of errors in generated code [61].

Prior work addresses parts of this problem. Lucassen et al. [55] developed a framework for evaluating user stories according to syntactic, semantic, and pragmatic criteria. Research on BDD shows that executable specifications can still suffer from duplication and weaknesses in readability, reuse, abstraction, and maintainability [13, 14]. More recently, Houck et al. [47] proposed CAFE(S), a conceptual framework identifying clarity, actionability, fidelity, efficiency, and security as desirable properties of agent context.

These contributions raise an urgent question for SDD in the age of AI: *Which qualities and practices characterize specification and planning artifacts intended to guide AI-assisted implementation?* To address this need, we introduce **EPIC**, an empirically grounded framework of 40 practices in ten quality dimensions for deciding what to tell coding agents and how to express it across specifications, plans, and tasks.

3 Methodology

We used a mixed-methods design [74] to develop and evaluate **EPIC**. We first identified and organized practices for addressing the quality of specification and planning artifacts in SDD (RQ1).

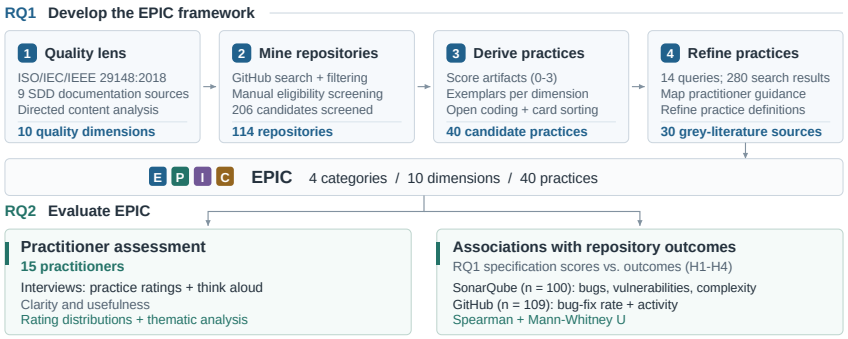

Fig. 1. Overview of the study design.

We then evaluated **EPIC** through practitioners' views of the framework's clarity and usefulness and examined associations between specification quality and indicators of code quality and development activity (RQ2). Figure 1 summarizes the study design.

3.1 RQ1. Developing the EPIC Framework

We developed **EPIC** in four steps. (1) We drew on ISO/IEC/IEEE 29148:2018 [1] and SDD tool guidance [41, 51] to establish ten specification quality dimensions, which formed an initial *quality lens*. (2) We collected specification and planning artifacts from open-source SDD repositories. (3) We assessed these artifacts against the quality lens and analyzed exemplars for each dimension to identify concrete practices for addressing it. This analysis yielded 40 candidate practices. (4) We examined practitioner guidance through a grey literature review [37] to examine their support for the practices, refine their definitions, and identify concerns not captured by the repository analysis.

3.1.1 Establishing a Specification Quality Lens. Requirements engineering standards define characteristics used in empirical research and quality assurance to assess requirements quality [28, 60]. Prior studies have also drawn on ISO-defined characteristics to examine requirements and software quality, including in AI-assisted development [31, 66]. Following this approach, we selected ISO/IEC/IEEE 29148:2018, a requirements engineering standard that defines the characteristics of good software requirements.

Incorporating SDD tool guidance. To interpret these characteristics in the context of SDD, we complemented them with documentation and specification templates from popular SDD kits (N=9) (i.e., GitHub Spec Kit [41], Kiro [51]). We used these documents to understand how requirements inform design decisions and implementation tasks in SDD, and what information agents need at each stage.

Coding and reconciling quality expectations. We conducted a directed qualitative content analysis [48], using the standard's requirements-quality characteristics and definitions as the initial codebook. Four researchers independently applied this codebook to the SDD kits (N=9) documentation, treating each passage expressing a quality expectation or specification practice as a coding unit. They applied existing codes to matching passages and assigned provisional codes to additional quality concerns in SDD. In weekly meetings, they compared their coding and reconciled differences through negotiated agreement [16].

Defining the ten quality dimensions. This analysis yielded ten quality dimensions for specification and planning artifacts in SDD. Seven retained characteristics are named in the ISO/IEC/IEEE 29148:2018: *Unambiguous*, *Singular*, *Verifiable*, *Conforming*, *Complete*, *Consistent*, and *Comprehensible* [1]. The remaining three adapt or synthesize the standard and tool guidance

for SDD. *Dependency Control* adapts *Feasible* to focus on the documented assumptions, dependencies, and constraints that condition whether requirements can be realized [1, 41]. *Scope Discipline* combines guidance on requirements abstraction and the separation of requirements from implementation decisions [1, 41]. *Implementation-Planning Information* captures the planning and task information emphasized by the SDD tools [41, 51], informed by the standard's provisions for assumptions, dependencies, and requirements allocation [1]. See the definitions of the ten quality dimensions in [7].

3.1.2 Mining OSS Repositories for SDD Artifacts. The quality dimensions describe what a good specification achieves, but not how a specification achieves it. To identify practices for addressing those properties, we needed to understand how SDD projects documented requirements and implementation guidance. We planned to use the ten quality dimensions to assess these artifacts and select examples from which to derive practices.

Discovering candidate repositories. We first collected specification and planning artifacts from GitHub. Because SDD workflows vary in their tools, filenames, and directory structures [17, 71], we used two types of discovery signals to identify candidate repositories: (1) specification-related filenames and framework-specific structures. Filenames such as `spec.md`, `requirement.md`, and `PRD.md` helped identify candidates that might use SDD without a dedicated framework. (2) We also searched for structures associated with Spec Kit and OpenSpec. For Spec Kit, these included the `.specify/` directory and artifacts such as `spec.md`, `plan.md`, `constitution.md`, and task files. For OpenSpec, they included `openspec/changes` and artifacts such as `proposal.md`, `design.md`, and specification files. We executed the searches through the GitHub Code Search API on April 25, 2026. This step yielded 850 candidate repositories.

Filtering and screening candidate repositories. We narrowed the candidate set through repository-level filtering and manual eligibility screening [9, 58]. We applied three filters adapted from prior studies. First, we removed forks and duplicate repositories to reduce duplication in the sample [29, 50, 62]. Second, we retained repositories whose relevant specification artifacts were written in English [2]. Third, we required at least ten GitHub stars as a minimum signal of community interest inspired by existing studies [17, 58]. This step yielded 206 repositories.

We then screened the remaining repositories for evidence of SDD use. Before screening, four authors defined the inclusion criterion: a repository was eligible if it contained specification artifacts intended to guide or constrain an AI agent's implementation. Two authors independently screened the remaining repositories by inspecting their documentation and artifacts, then resolved disagreements through negotiated agreement [38]. Of the 206 repositories screened, 114 met the criterion and formed the final dataset.

3.1.3 Deriving Candidate Specification Practices from SDD Artifacts. Using the SDD repository dataset, we examined how specification and planning artifacts addressed the ten quality dimensions in practice. For each dimension, we selected artifacts with strong evidence of that quality and analyzed them to identify the practices they used. The analysis had five steps:

① *Operationalizing the quality dimensions.* To assess repositories, we developed a four-point rubric based on the definitions and descriptions of the ten quality dimensions. We scored each repository on each dimension using evidence from its specification and planning artifacts: 0 indicated little or no evidence of the quality, 1 partial evidence, 2 mostly consistent evidence, and 3 consistent evidence throughout the artifacts. For example, for *Singular* (E2), a repository scored 0 when requirements frequently bundled multiple capabilities or concerns into one statement; 1 when some requirements were singular but compound statements remained common; 2 when most requirements expressed a single capability, characteristic, constraint, or quality, with occasional

compound statements; and 3 when all requirements did so consistently. The full rubric is available in the replication package [8].

② *Calibrating the application of the rubric.* Two researchers then independently assessed the same repositories in two calibration rounds and discussed every disagreement after each round. We first calibrated the rubric on approximately 10% of the dataset (13 repositories). After discussing scoring differences, we conducted a second round on approximately another 20% (23 repositories) to check whether the researchers could apply the rubric consistently to new repositories. These rounds yielded 130 and 230 paired dimension ratings.

We assessed agreement using exact agreement, quadratic-weighted Cohen's κ_w, and ordinal Krippendorff's α [20]. The latter two measures account for chance agreement and distinguish smaller from larger disagreements on the ordinal scale. We interpreted κ_w using Landis and Koch's categories [53]. In the first round, the researchers agreed on 104 of 130 ratings (80.0%), and no disagreement exceeded one scale point. Weighted agreement was almost perfect ($\kappa_w = 0.82$; $\alpha = 0.94$). In the second round, exact agreement was 83%, with substantial weighted agreement ($\kappa_w = 0.78$; $\alpha = 0.80$). These results supported proceeding with the division and scoring of the remaining repositories.

③ *Scoring the remaining repositories independently.* The two researchers divided the remaining 78 repositories between them. Each researcher scored their assigned repositories and discussed difficult or ambiguous cases with the other researcher.

④ *Selecting exemplars for each dimension.* During the analysis, we observed a repository could be an exemplar for *Unambiguous* without being one for *Complete*; we therefore selected exemplars separately for each dimension. For each dimension, we selected the top-scoring 20% of repositories and used their artifacts as examples for the subsequent analysis [9]. When repositories shared the cutoff score, we included all of them. The step yielded between 28 and 37 exemplar repositories per dimension.

⑤ *Deriving practices from the exemplars.* For each dimension, two researchers independently coded selected exemplars to identify the specification practices they contained. Using open coding with constant comparison [70], they compared new passages with those already coded, reusing a code when it described the same practice and creating a new code when no existing code fit. They met weekly to compare their coding, merge duplicate codes, and resolve disagreements through negotiated agreement [38]. Across the ten dimensions, they coded 5,675 segments.

The researchers then used card sorting to group codes describing similar ways of addressing each dimension [84]. They clarified the boundaries between groups and named and defined each group as a specification practice under that dimension. This process yielded 40 practices, each describing a recurring way the exemplar artifacts addressed a quality dimension.

3.1.4 Refining Practices with Practitioner Guidance from Grey Literature. To validate and refine the practices derived from repositories, we reviewed practitioner guidance in blogs, technical guides, framework documentation, and repositories [2, 17, 36, 69]. We examined whether these sources recommended the practices, identified missing concerns, or suggested revisions, drawing on their explanations of why and how the practices should be applied. We followed guidelines for grey literature research (GLR) in software engineering [37].

① *Searching for sources.* To identify relevant grey literature, we drew on terminology from the Spec Kit and Kiro documentation [41, 51] to formulate 14 search queries, organized into three groups. The first targeted SDD terminology, including "spec-driven development" and "specification-driven development," alone or combined with "LLM." The second targeted frameworks and artifacts, including Spec Kit, OpenSpec, `AGENTS.md`, `CLAUDE.md`, and `copilot-instructions.md`. The third restricted searches to GitHub to locate repository-based guidance and discussions. We collected the

first 20 Google results for each query, following an approach also used in prior software engineering GLR [27, 72]. The step yielded 280 artifacts.

② *Screening sources.* Before screening, we defined three inclusion and five exclusion criteria. A source had to meet all three inclusion criteria: (1) address writing or using specifications for AI-assisted development (*topic relevance*); (2) recommend concrete practices or provide examples (*actionable content*); and (3) make its full text freely available (*accessibility*). We excluded sources that (1) used "specification" in a sense unrelated to guiding AI agents, such as test, hardware, or API specifications; (2) mentioned SDD or AI coding tools only in passing, or consisted solely of news, announcements, marketing, or installation instructions; (3) duplicated or republished a source already retrieved; (4) were inaccessible (broken, paywalled, or login-only) or non-textual without a transcript; or (5) were peer-reviewed academic papers, which fall outside grey literature. This process yielded 30 artifacts.

③ *Mapping guidance to practices.* We then conducted a directed qualitative content analysis using the 40 practices as the initial codebook [48]. Two researchers independently coded artifacts focusing on specification-writing practices, recommendations, or examples from each source, mapping each passage to the practices it supported. They then compared their coding and resolved disagreements through negotiated agreement [38]. The review did not lead to additional practices or dimensions. Each concern identified in the 30 sources either mapped to an existing practice or fell outside the study's scope. The sources provide explanations of why practices matter and how to apply them. We used these explanations to revise the definitions of practices, clarifying what each practice requires and how a specification author can address it (see the GLR mapping in [7]).

3.1.5 Assembling the EPIC Framework. We next organized the dimensions and practices into a framework. Four researchers compared each dimension's definition and associated practices, then grouped the dimensions by the questions they address about a specification. They resolved disagreements through negotiated agreement [38]. The resulting **EPIC framework** (Figure 2) comprises four categories, ten quality dimensions, and 40 specification practices.

3.2 RQ2. Evaluating the EPIC Framework

3.2.1 Practitioner Evaluation of Clarity and Usefulness. We first assessed how practitioners interpreted **EPIC**'s practices, whether they considered them useful for constructing specifications, and what they would change. Inspired by the established approaches to taxonomy evaluation [63, 77], we conducted individual semi-structured interviews [29, 31].

Designing and piloting the interviews. Each interview lasted approximately 60-90 minutes and began with a brief demographic questionnaire. We then asked participants about their use of SDD, including their development practices, motivations, challenges, and learning experiences. These questions provided context for interpreting their subsequent assessments.

Participants were then asked to review the framework's 10 dimensions and 40 metrics through a custom web application [7]. Each metric appeared on a card with an explanation and a real example from GitHub exemplars. For each metric, participants rated their agreement with the statement on a five-point Likert scale ranging from strongly disagree to strongly agree.

Throughout the interview, participants were asked to think aloud, explaining their interpretation of each metric and the reasoning behind their rating. Those unfamiliar with thinking aloud completed a brief practice exercise. We used follow-up questions to explore unclear wording, practical usefulness, overlap between metrics, and missing practices. Participants could suggest modifications and share examples from specifications they had written or reviewed. The interview concluded with questions about the framework's overall structure and how well it reflected SDD practice.

To encourage feedback and reduce acquiescence bias [44], we emphasized that we were evaluating the framework, and explicitly welcomed disagreement and criticism. Dimensions appeared in

randomized order to distribute potential ordering and fatigue effects across the framework's dimensions [35]. Interviews were audio-recorded and transcribed with participants' consent.

Before conducting the study, we performed three pilot interviews to assess the interview protocol and evaluation instrument, including the clarity of the instructions and the time required to review the framework. Based on the pilot feedback, we revised the interview flow and rephrased instructions and questions to make the interview easier to follow.

Participant recruitment and data analysis. We recruited software professionals from our professional network and used snowball sampling to recruit additional participants through their referrals. To ensure that participants have SDD experience, we first emailed potential participants to ask whether they had used SDD. We then invited only those who confirmed such experience to take part in the study. The study was approved by our university's IRB, and participants were offered a $50 gift card. In total, we recruited 15 participants: seven backend or full-stack developers, two data/ML engineers, a QA engineer, a system architect, a project manager, an electrical engineer, and two software researchers. Fourteen had at least six years of development experience, eleven used specifications or structured context to guide AI coding tools often or very often, and twelve had used AI tools with written specifications for six months or more (see demographics in [8]).

To understand why participants rated each practice as they did, we analyzed their explanations using inductive coding [15] and card sorting [84]. We organized the transcripts by practice, keeping each explanation linked to its corresponding rating. Two researchers independently developed codes from participants' explanations, then compared their coding and resolved disagreements through negotiated agreement [38]. They used card sorting to group related codes into themes capturing participants' reasons for agreement and disagreement [84]. After analyzing the first 13 interviews, the results were approaching saturation and we stopped recruitment. We then completed and analyzed two additional interviews that had already been scheduled to confirm this assessment. Neither yielded new substantive feedback on the framework, satisfying our saturation criterion of two consecutive interviews without new findings.

3.2.2 Specification Quality and Software Outcomes. We also examined whether the specification qualities underlying **EPIC** were associated with code quality and development activity. We focused on code quality and development activity because they address two fundamental concerns in software engineering: building dependable software and sustaining its maintenance and evolution [5, 33]. Code quality provides evidence about the implementation, while development activity reflects the corrective work and contributor participation through which software evolves.

To conduct the analysis, we returned to the 114 GitHub repositories identified in RQ1 as using specifications to guide AI agents' implementation work. Each repository had received a score from 0 to 3 on each of the ten quality dimensions, based on evidence in its specification and planning artifacts. We summed these scores to obtain an overall specification-quality score between 0 and 30.

During the analysis, we had to disregard 5 repositories. 2 of them [R60, R599] were deleted or moved to a different location by the time we performed the analysis. We also faced cloning problems with 3 others [R63, R156, R236], which did not allow us to perform the analysis. Hence, we perform the SonarQube and GitHub analysis in 109 repositories.

(1) Examining code quality: reliability, security, and structural maintainability. In a study of Cursor adoption, He et al. found temporary gains in development velocity but persistent increases in static-analysis warnings and code complexity [43]. We therefore examined whether specification quality was associated with reliability, security, and structural maintainability. Explicit descriptions of expected behavior and edge cases can improve the functional correctness of generated code [83]. Stating security expectations may similarly help agents avoid unsafe assumptions, addressing concerns raised by vulnerabilities in Copilot-generated code [67]. Clear scope and implementation guidance may also reduce ad hoc decisions and unnecessary branching, particularly when agents

fail to recognize underspecified instructions [79]. We therefore hypothesize that *higher specification quality is associated with lower bug density (H1), lower vulnerability density (H2), and lower cyclomatic complexity per KLOC (H3).*

We attempted SonarQube analysis on 109 repositories. However, 9 repositories faced SonarQube limitations that resulted in incorrect or incomplete information: 4/9 could not be measured by SonarQube (2 because the analysis failed and 2 because they contained source code that was not analyzable) and 5/9 had fewer than 1,000 NCLOC. For that reason, for H1–H3 analysis, we used 100 repositories. The analysis was conducted through a local SonarQube Community Build server (version 26.9) running in Multi-Quality Rule (MQR) mode, using SonarScanner CLI 8.0.1, the server's bundled analyzers (e.g., Python 5.31, JavaScript/TypeScript 13.8, Java 8.41, and Go 1.43), and the built-in *Sonar way* quality profile for each language. For each repository, we made a clone of its default branch and analyzed the latest commit; analyses ran on September 12th, 2026.

(2) Examining corrective work and development activity. Beyond code quality, software depends on continued contributions to address problems, review changes, and develop new capabilities [5]. Specifications may support this work by giving contributors a shared understanding of what to build and how their tasks relate. Explicit requirements may also reduce corrective work caused by misunderstandings or omissions. We therefore hypothesize that *higher specification quality is associated with a lower bug-fix commit rate and higher development activity (H4).*

To test H4, we collected repository metrics for 109 repositories through the GitHub REST API, including counts of pull requests, issues, contributors, merged pull requests, and commits. To examine corrective work, we used the bug-fix commit rate, motivated by prior research on corrective maintenance [6]. We calculated this rate as the share of scanned commits whose messages began with a fix pattern, such as `fix:`, `bugfix`, or `hotfix`, restricting the calculation to repositories with at least 20 scanned commits. The median repository had 136 pull requests, 50 issues, six contributors, and 771 commits on its default branch. Among repositories with at least 20 commits, the median share of bug-fix commits was 14.9% (see [8] for details of each metric analyzed).

(3) Testing the hypotheses. We used two complementary analyses to test H1–H4. First, we computed Spearman's rank correlation between the overall specification-quality score and each outcome measure across the full score range. Second, we compared outcomes between repositories with higher and lower specification quality. We divided the observed score distribution into tertiles, each intended to contain approximately one-third of the repositories, and compared the outer bands [22, 40]. Such comparisons between upper and lower score tertiles have been used in empirical software engineering [22]. The observed boundaries yielded bottom (≤ 18), middle (19–22), and top (≥ 23) bands, with at least five score points separating the top and bottom bands. We kept repositories tied at each cutoff together in the corresponding outer band, so group sizes could differ. We compared outcomes between the top and bottom bands using the Mann–Whitney U test and reported Cliff's δ as the effect size.

4 Results

In this section, we first present the EPIC framework for characterizing the quality of specification and planning artifacts. We then report practitioners' assessments of the practices and the associations between specification quality and repository outcomes.

4.1 The EPIC Framework for Specification-Driven Development (SDD)

EPIC is a multidimensional framework designed to support the development of high-quality software with SDD. It provides practices for writing and reviewing the specification and planning artifacts that guide AI agents' implementation work. The framework, as shown in Figure 2, organizes ten quality dimensions and 40 specification practices into four complementary categories:

Expressive, **P**recise, **I**nterpretable, and **C**ompartmentalized. An interactive version is available at [7].

E: Expressive. An AI agent needs to know what is required and how its work will be checked. An expressive specification communicates this through requirements with a clear intended meaning (*Unambiguous*, E1), one identifiable obligation (*Singular*, E2), and explicit criteria for checking whether that obligation has been met (*Verifiable*, E3) [1].

E1. Unambiguity An unambiguous specification makes the intended meaning of each requirement clear and identifies where that meaning remains unresolved. Markers such as *[NEEDS CLARIFICATION]* identify unresolved decisions and show where further input is needed (E1.1) [B4, B26].[1] For settled decisions, explicit, measurable bounds state the expected behavior and the conditions under which it applies (E1.2). For example, one repository specified that "*Operators can query any SuzieQ-supported network table and receive results within 5 seconds for networks up to 500 devices*" (R441).[2] The five-second limit states the required response time, while the 500-device limit bounds the applicability. The same repository listed the supported network tables, making clear which tables the requirement covered (E1.3) [B27, B30]. Normative wording makes clear whether a statement requires, permits, or prohibits an action. For example, "*It MUST NOT touch the persistent* `MultiSweepService`*/scheduled sweep config*" (R573) states a binding prohibition. Consistent use of these terms and statement structures supports interpretation across requirements (E1.4). Explicit exclusions and verification boundaries further specify what each requirement and its checks cover (E1.5) [B9, B10].

E2. Singularity. A singular requirement states one capability, property, or constraint. Details that define the same capability can stay together. For example, one repository specified "*R7. Support max_depth parameter (default 8, max 15) to limit tree depth*" (R80). The default and maximum values define the depth limit, so they belong in the same requirement (E2.1). The repository recorded a scope exclusion separately: "*This feature does NOT capture visual layout or screenshots.*" Each statement can then be reviewed on its own. The other practices apply this approach to design facts, risks and assumptions, definitions and scenarios, and security properties (E2.2–E2.5) [B1, B29].

E3. Verifiability. A verifiable specification states what counts as satisfying a requirement, how satisfaction can be checked, what scenarios the checks must cover, and when verification is considered complete. For example, one repository documented "*Backend coverage ≥ 80% (from 54%)*" and supplied a command containing `--fail-under=80` (R224). The target states the minimum acceptable coverage (E3.1) [B1, B21, B28], while the command provides a way to check it (E3.2) [B7, B8]. The documents show how to check the target, but do not show whether the check was run or passed. More generally, a verifiable specification identifies the scenarios that its checks must cover (E3.3) [B5, B6] and the conditions for considering verification complete (E3.4) [B27, B30].

P: Precise. To plan and carry out implementation work, an agent needs to understand both the conditions on which requirements depend and the scope it is expected to address. This category connects explicit prerequisites and constraints (*Dependency Control*, P1) [1] with coverage of the intended requirements (*Complete*, P2) [1].

P1. Dependency Control. An agent needs to know which assumptions and prerequisites must hold for a requirement to be realized. Declaring assumptions and dependencies (P1.1) [B17, B19] and stating constraints and scope boundaries (P1.2) [B30] make these conditions explicit. Linking them to individual requirements or tasks places that information alongside the work it affects (P1.3). For example, one repository has a statement "*Dependencies: Task 1 (sequential - needs root to link properly)*" that maps to a task (R102). The entry identifies the prerequisite and explains why it must

[1]The prefix B identifies grey-literature sources listed in the supplementary material [7].
[2]Repository identifiers refer to entries in the *Repositories* page; see [7].

come first, providing a rationale for sequencing the work. Dependency verification and failure handling further specify how these conditions should be checked and what happens when they are not met (P1.4).

P2. Completeness. Completeness concerns whether the artifact set accounts for the requirements within the feature's stated scope, including functional and user requirements (P2.1) [B9, B13] and applicable exceptional behavior (P2.2) [B2, B5]. Scope exclusions identify what falls outside that boundary; missing or unresolved requirements within it indicate remaining gaps. For example, one repository included an edge-case table pairing "*Degenerate/near-zero triangles*" with "*Filter or robust epsilon*" (R164). The entry identifies a case the implementation needs to address, although the choice of handling remains open. Coverage also includes technical, operational, and quality constraints (P2.3) [B15, B22]. Supporting plans record dependencies and how the work will be delivered (P2.4) [B7]. Tracking unresolved specification items makes clear which parts of the feature still need decisions (P2.5) [B6, B30].

I: Interpretable. An agent working across specifications, plans, and task descriptions needs to understand how the information fits together. Recognizable conventions help it locate requirements (*Conforming*, I1) [1], compatible terminology and content preserve meaning across artifacts (*Consistent*, I2) [1], and contextual guidance explains how the details relate to the intended product (*Comprehensible*, I3) [1].

I1. Conformance. Conformance asks whether the artifacts follow the schema, template, or document conventions adopted for the project. Standardized structures establish where information belongs (I1.1) [B7], while stable, unique identifiers allow individual requirements to be referenced across documents (I1.2) [R385, R400]. Explicit template declarations identify the conventions the artifacts are expected to follow (I1.3) [R442, R446]. For example, one OpenSpec repository declared `schema: spec-driven` (R72). This gives a reader a named schema against which to check the artifacts.

I2. Consistency. As an agent moves between documents, the meaning of a requirement needs to remain the same. Terms and identifiers should refer to the same concepts, and descriptions of the same behavior under the same conditions should agree. Shared terminology and definitions establish these meanings (I2.1) [R72, R383], while uniform notation keeps their expression consistent (I2.2) [R385, R441, R442]. In the same OpenSpec repository, seven requirements contained 35 `SHALL` and four `SHALL NOT` statements with no other modal variants. Each of its 21 scenarios contained one `GIVEN/WHEN/THEN` triple, illustrating consistency in normative wording and scenario structure.

Consistency also connects the specification to the planned implementation. One repository used the response fields `checks_completed` and `checks_skipped` identically in its specification, error cases, and implementation plan (R80). The matching names preserve the correspondence between the interface being specified and the one being planned (I2.3) [B12, B16]. Repeating the same section organization across related artifacts provides another form of consistency (I2.4) [B30].

I3. Comprehensibility. The artifact set also needs to explain what is being built and how to navigate its description. Sections organized by purpose give readers a way into the details. For example, one repository organized its product requirements document into fifteen numbered sections, moving from the problem statement to non-goals (R166; I3.1) [B30]. The same document introduced the product with a short statement: "*This is a prompt flow editor, not a chat app.*" This tells the reader what the requirements describe and rules out a different interpretation of the product (I3.3) [B3, B30]. When specifications, plans, and tasks occupy separate files, cross-document links and explanations of their relationships provide a route through the artifact set (I3.2) [B23, B24].

C: Compartmentalized. An agent needs to distinguish what the product must do from the decisions and tasks involved in building it. This category connects a specification focused on required behavior (*Scope Discipline*, C1) [1] with supporting artifacts that guide implementation (*Implementation-Planning Information*, C2) [32, 41, 52].

C1. Scope Discipline. The product specification describes the capabilities users need and the constraints the product must satisfy. For example, one repository stated, "*As a network engineer, I want to query what the network looked like at a specific point in time*" (R441). This describes the capability from the user's perspective (C1.1) [B2, B20]. In the analyzed examples, design decisions, technology stacks, and task lists appeared in separate plan and task artifacts (C1.2) [B13, B18]. This gives implementation details a place while keeping the required behavior identifiable. Technical information can still belong in the product specification: a necessary compatibility, interface, or operating constraint is a requirement the implementation must satisfy.

C2. Implementation-Planning Information. Implementation-planning information concerns the guidance recorded in supporting plan and task artifacts. This includes the technical context in which the work will be implemented (C2.1) [R100, R165], what individual tasks are intended to produce (C2.2) [B9, B11], and the data structures or interface contracts the implementation must respect (C2.4) [B10, B14]. For example, one repository supplied typed interfaces for stored entities, making the planned data structures and their types explicit (R100).

Traceability keeps these separate artifacts connected. *Traceable Transition to Implementation* (C1.3) [B3, B8] and *Requirements-to-Implementation Traceability* (C2.3) [R441, R442] link requirements to the work intended to implement them. A task can then be read alongside the requirement it is meant to satisfy, keeping its purpose visible even when the implementation details appear in a different document.

4.2 RQ2. Evaluating the EPIC Framework

After developing **EPIC**, we evaluated its practical relevance in two ways. First, we examined how practitioners interpreted **EPIC**'s practices. We then assessed whether repositories with higher specification-quality scores also showed better code quality and greater development activity.

*4.2.1 Practitioner Assessment of **EPIC**'s Usefulness.* Overall, every metric in **EPIC**'s ten dimensions received agreement or strong agreement from a majority of participants as shown in Figure 2.

Expressive. Participants endorsed the *Expressive* practices for clarifying requirements (E1), keeping them singular (E2), and defining how they should be verified (E3) (Figure 2).

Unambiguous (E1): Nine of 15 participants strongly agreed with *explicit clarification-seeking* (E1.1). P2[3] explained that when an agent faces an unresolved decision it may "*just assume something*". For decisions already made, 14 participants agreed with *normative & structural consistency* (E1.4). P4 explained that words such as *should* or *may* leave room for another interpretation: "*the AI could say, okay, it should, so I'm going to go through the other way*". Their team therefore writes "*you must do this, you must do that, so it has no open other interpretations*". Thirteen participants also agreed with *explicit boundaries* (E1.5). P4 described how they kept the requested work within scope: "*do not embrace the world. You have to focus on these tasks and only do this*".

Singular (E2): Eleven participants agreed with *isolated risk & edge-case entries* (E2.3). P2 explained that when each case has "*its own bullet points [...] the LLM tends to understand a little more*". Nine participants strongly agreed with *singular definitions & scenarios* (E2.4). P2 explained that keeping scenarios separate "*really helps the agent to not be lost of what to do and how to do*". The connection between separation and checking also appeared in comments on *singular security & privacy properties* (E2.5), which received ten strongly agree ratings. P5 explained that recording these properties separately could allow the agent to check each one individually.

Verifiable (E3): Twelve participants agreed with *measurable outcomes* (E3.1). P10 explained that "*if [agent] doesn't know it's done, it's going to be impossible for it to stop the harness loop*". Twelve participants also agreed with *verification methods* (E3.2); "*they say, yeah, 780 tests have passed. I don't really know anything about those tests and what they cover*" (P3). Thirteen participants agreed

[3]Interview participants are identified as P1–P15; see [8] for their demographics.

EPIC Practitioner assessment of specification practices

15 participants per practice / 10 quality dimensions / 40 practices

Ratings: 1 = strongly disagree; 2 = disagree; 3 = neutral; 4 = agree; 5 = strongly agree.

E Expressive

		1	2	3	4	5	AGREE
E1	**Unambiguous**						
E1.1	Explicit Clarification-Seeking	-	1	2	3	9	12/15
E1.2	Quantified / Measurable Requirements	1	-	4	5	5	10/15
E1.3	Bounded Vocabulary & Enumerated Definitions	1	-	1	5	8	13/15
E1.4	Normative & Structural Consistency	1	-	-	8	6	14/15
E1.5	Explicit Scope & Verification Boundaries	-	1	1	8	5	13/15
E2	**Singular**						
E2.1	Atomic Functional & Behavioral Statements	2	1	4	2	6	8/15
E2.2	Singular Architecture & Design Facts	1	1	3	4	6	10/15
E2.3	Isolated Risk, Assumption, Edge-Case & Constraint Entries	1	-	3	6	5	11/15
E2.4	Singular Definitional, Scenario & Relational Statements	1	1	2	2	9	11/15
E2.5	Singular Security & Privacy Properties	1	1	1	2	10	12/15
E3	**Verifiable**						
E3.1	Verification Criteria & Measurable Outcomes	-	-	3	6	6	12/15
E3.2	Verification Scenarios & Validation Methods	-	2	1	4	8	12/15
E3.3	Comprehensive Verification Coverage	-	2	2	2	9	11/15
E3.4	Verification Completion & Readiness	-	1	1	4	9	13/15

I Interpretable

		1	2	3	4	5	AGREE
I1	**Conforming**						
I1.1	Standardized Document Structure	-	-	3	4	8	12/15
I1.2	Stable, Unique Requirement Identifiers	-	1	1	4	9	13/15
I1.3	Explicit Template Declaration & Enforcement	-	1	1	3	10	13/15
I2	**Consistent**						
I2.1	Standardized Terminology & Concept Definitions	1	-	-	3	11	14/15
I2.2	Uniform Identifiers & Notation	1	-	-	4	10	14/15
I2.3	Cross-Artifact Alignment & Traceability	1	-	2	5	7	12/15
I2.4	Consistent Organizational Structure	1	1	-	4	9	13/15
I3	**Comprehensible**						
I3.1	Structured Document Organization	1	-	2	1	11	12/15
I3.2	Cross-Document Context & Navigation	1	-	2	5	7	12/15
I3.3	Comprehension Support	1	-	1	5	8	13/15

P Precise

		1	2	3	4	5	AGREE
P1	**Dependency Control**						
P1.1	Declared Assumptions & Dependencies	-	-	3	5	7	12/15
P1.2	Explicit Constraint & Scope Boundaries	-	-	3	2	10	12/15
P1.3	Requirement/Task-Level Dependency & Risk Traceability	-	1	3	3	8	11/15
P1.4	Dependency Verification & Failure-Mode Handling	-	3	1	3	8	11/15
P2	**Complete**						
P2.1	Functional and User Requirement Coverage	-	1	2	4	8	12/15
P2.2	Exceptional Behavior & Scope Definition	-	-	1	7	7	14/15
P2.3	Technical, Operational & Quality Constraints	-	1	-	6	8	14/15
P2.4	Dependency & Delivery Planning	-	-	6	4	5	9/15
P2.5	Specification Completeness Management	-	1	3	7	4	11/15

C Compartmentalized

		1	2	3	4	5	AGREE
C1	**Scope Discipline**						
C1.1	Behavioral Product Specification	1	1	2	3	8	11/15
C1.2	Separation of Product & Project Concerns	1	2	2	3	7	10/15
C1.3	Traceable Transition to Implementation	1	-	3	2	9	11/15
C2	**Implementation-Planning Information**						
C2.1	Technical Context & Implementation Foundations	-	1	-	3	11	14/15
C2.2	Implementation Planning & Task Organization	-	1	2	3	9	12/15
C2.3	Requirements-to-Implementation Traceability	-	4	-	4	7	11/15
C2.4	Implementation Data Models & Evolution Planning	-	1	2	6	6	12/15

Cells show participant counts; darker cells indicate more ratings. - = 0. Agree = ratings 4 + 5.

Fig. 2. The EPIC framework, organized into four categories, ten quality dimensions, and 40 practices, alongside evaluations from 15 practitioners. The columns to the right of each metric show the number of participants rating, from 1 (strongly disagree) to 5 (strongly agree).

with *completion conditions* (E3.4). P2 described instances in which an agent stopped and announced that it had finished while leaving work incomplete. Stating completion conditions gave participants a basis for checking such claims and deciding whether the work could proceed. P9 explained that developers should always provide a "*go signal or no go signal*".

Participants explained their lower ratings. For example, P3 explained that "*now the AI figures this out*" when discussing quantified requirements (E1.2), described agents as "*trained to be really guarded about security and privacy*" when discussing separate security and privacy properties (E2.5), and felt that splitting requirements could become "*way too much to track*". However, Pearce et al. [67] found vulnerabilities in approximately 40% of 1,689 programs generated across 89 security-relevant scenarios using an early version of Copilot.

Precise. A majority of participants agreed with every practice in this category, with five metrics receiving agreement from at least 12 of 15 participants (Figure 2).

Dependency Control (P1): Ten participants strongly agreed with *explicit constraints & scope boundaries* (P1.2). P12 explained that explicitly stating constraints "*prevents the design from going off the rails and requiring significant rework*", recalling cases where they had "*ended up having to completely redo like big subsystems*". Eleven participants agreed with *dependency verification & failure-mode handling* (P1.4), including eight who strongly agreed. Their concern was that an implementation could assume its dependencies always work. P4, whose system relied on external APIs, wanted it stated explicitly "*what needs to be done when it fails, when it's out of service*".

Complete (P2): Exceptional behavior & scope definition (P2.2) and *technical, operational & quality constraints* (P2.3) each received agreement from 14 participants. P9 explained: "*if you think there is enough, your customer will come and break the product in a way you've never seen*". Explicit exclusions also helped define the coverage needed: stating what falls outside the feature "*helps a lot to funnel through what you actually need*" (P6). For *technical, operational & quality constraints* (P2.3), P10 explained that different parts of a system may need different priorities. For example, "*a billing system might have harder requirements around accuracy and replayability*", whereas a front-end component may have different needs. These expectations therefore need to be stated; otherwise, the agent is "*making a decision even if I don't*" (P10). P2.5 (*specification completeness management*) was endorsed by 11 participants, who treated the record of decisions as context the agent can reuse. P11 kept a daily "*log and make entries in a markdown, and then query against that with the LLM*", which helped "*inject the context and history that I was looking for*".

Participants also explained their lower ratings in this category. Three participants disagreed with *dependency verification & failure-mode handling* (P1.4). P9 explained that "*an agent will typically ask what you want to do when something fails*", suggesting that failure-handling decisions could be addressed when the problem arose. However, this expectation depends on the agent recognizing when it needs guidance and asking for it. Studies of coding agents show that detecting missing information and seeking clarification vary with the model, prompting, and agent configuration [26, 79]. For *dependency & delivery planning* (P2.4), six participants were neutral. P10 questioned the need for detailed planning early in a project, when "*you just want to be able to run it right now*". This comment points to a question of timing: planning the delivery process may seem less necessary when the immediate goal is to get an initial implementation running.

Interpretable. This category received the highest proportion of agreement (Figure 2).

Conforming (I1): Thirteen participants agreed with *stable, unique requirement identifiers* (I1.2). P5 used requirement numbers to trace product requirements into specifications and check that each had been addressed. P4's team also kept retired requirements visible so that a changed requirement "*does not silently disappear*". Ten participants strongly agreed with *explicit template declaration & enforcement* (I1.3). P7 explained why declaring a convention alone may be insufficient: "*The checks that you mechanize and make blockers on your process are the ones that actually get followed*".

Consistent (I2): Fourteen participants agreed with *standardized terminology & concept definitions* (I2.1), including 11 who strongly agreed. P2 described agents using different names for the same concept in different places. Consistency also mattered when information was distributed across documents. Twelve participants agreed with *cross-artifact alignment & traceability* (I2.3). P5 described a message schema maintained in one repository and its implementation in another.

Comprehensible (I3): Eleven participants strongly agreed with *structured document organization* (I3.1). P10 explained that a clear structure helps both agents and developers locate relevant information: "*both for humans to search, but also agents can go look at the table of contents and then go find the important piece*". Beyond finding information, participants wanted guidance on how to use it. Thirteen agreed with *comprehension support* (I3.3). P2 explained that examples and guidance help "*the human that are reading it, but also helps the agent to know how to perform*".

All seven ratings of 1 came from P3, who expressed confidence that agents could handle consistency: "*in the AI world, it doesn't matter*". Research on trust in automation provides a possible explanation for such confidence. Users may initially consider automated systems trustworthy, but their confidence can decline after observing errors [24, 25].

Compartmentalized. Participants described two needs: keeping the business problem visible (C1) and giving agents enough technical direction to implement it (C2) (Figure 2).

Scope Discipline (C1): Eleven participants agreed with *behavioral product specification* (C1.1). P5 explained: "*When you mix those two things together, it becomes very hard to understand, like*

Table 1. Median code quality and activity by specification-quality band (n = 36/24/40 for SonarQube; 40/26/43 for GitHub). Arrows: hypothesized directions; Δ: relative top–bottom difference. p, δ: one-sided Mann–Whitney test and Cliff's effect size; ρ, p_ρ: Spearman correlation and one-sided p-value. H4 p-values are Benjamini–Hochberg-adjusted within each six-test family. Bold: $p < 0.05$.

ID	Measure	Specification-quality band			Top vs. bottom			Spearman	
		Bottom (≤ 18)	**Middle** (19–22)	**Top** (≥ 23)	Δ	p	δ	ρ	p_ρ
H1	Bugs/KLOC ↓	0.248	0.278	0.235	−5.1%	0.101	−0.171	−0.183	**0.035**
H2	Vulnerabilities/KLOC ↓	0.178	0.274	0.092	−48.0%	**0.016**	−0.286	−0.134	0.092
H3	Complexity/KLOC ↓	162.3	161.7	137.2	−15.4%	**0.001**	−0.419	−0.206	**0.021**
H4.A	Bug-fix commit rate ↓	20.4%	20.0%	11.8%	−42.4%	**0.034**	−0.333	−0.210	**0.046**
H4.B	Pull requests ↑	89.0	210.0	202.5	+127.5%	**0.036**	+0.241	+0.170	**0.046**
H4.C	PRs merged ↑	56.0	186.5	152.0	+171.4%	**0.036**	+0.260	+0.189	**0.046**
H4.D	Issues ↑	22.0	107.5	58.0	+163.6%	**0.036**	+0.229	+0.161	**0.048**
H4.E	Contributors ↑	3.0	5.5	12.5	+316.7%	**0.036**	+0.241	+0.201	**0.046**
H4.F	Commits ↑	439.0	917.5	942.0	+114.6%	**0.036**	+0.249	+0.170	**0.046**

what is the actual business problem that you're trying to solve". Participants also described where implementation guidance belonged. For *separation of product & project concerns* (C1.2), P2 placed that guidance in agent configuration, using "*some skills and some rules that say how*".

Implementation-Planning Information (C2): Fourteen participants agreed with *technical context & implementation foundations* (C2.1), including 11 who strongly agreed. P15 explained that without this context, an agent "*can give the different technology solution for you*". Twelve participants agreed with *implementation data models & evolution planning* (C2.4). P2 described what happened when their team specified only the database: the generated entities "*end up being like a direct copy of the database*", "*because the agents tried to copy, because it's the reference it has*". Such a case illustrates how agents may use an available reference to fill gaps in the intended data model. Four participants disagreed with *requirements-to-implementation traceability* (C2.3). P9 raised a concern about reviewing the volume of generated material: "*nobody will be interested in reading all the things that agents created*". Automated checks could help address this concern. For example, P5 described using a custom agent skill to identify requirements that a codebase had not yet implemented.

4.2.2 Associations with Code Quality and Development Activity. Table 1 summarizes the associations between specification quality, code-quality indicators, and development activity. We observed that higher specification quality was associated with lower values on all three code-quality indicators, but the strength of the evidence varied.

H1. Reliability was associated with specification quality across the full sample: repositories with higher scores tended to have fewer bugs per KLOC ($\rho = -0.183$, $p = 0.035$). However, the top and bottom bands had similar median bug densities (0.235 vs. 0.248), and their difference was not significant ($p = 0.101$), so H1 was supported only in the full-sample analysis. *H2. Vulnerabilities* was lower in repositories with the highest specification scores. Top-band repositories had roughly half the median vulnerability density of bottom-band repositories (0.092 vs. 0.178 per KLOC; $p = 0.016$, $\delta = -0.286$). This difference did not extend to a significant correlation ($\rho = -0.134$, $p = 0.092$), so H2 was supported only in the top-versus-bottom comparison. *H3. Structural maintainability* showed the most consistent association with specification quality. Top-band repositories had 15.4% lower median cyclomatic complexity per KLOC (137.2 vs. 162.3; $p = 0.001$), with a medium effect size ($\delta = -0.419$). Complexity was also negatively correlated with specification score ($\rho = -0.206$, $p = 0.021$), supporting H3 in both analyses. *H4. Repository activity* was higher in higher-score repositories. Top-band repositories had more pull requests, merged pull requests, issues, commits,

and contributors, including a median of 12.5 contributors compared with 3 in the bottom band. All five activities were significant in both analyses after BH correction ($p_{BH} \leq 0.048$). Bug-fix commits accounted for 11.8% of scanned commits in the top band versus 20.4% in the bottom band ($p_{BH} = 0.034$), and their share decreased with specification score ($\rho = -0.210$, $p_{BH} = 0.046$).

5 Discussion

What EPIC Adds to Requirements Quality. Requirements engineering has long established expectations for specifying what software must do and how its behavior should be verified [1]. Our results show that these expectations still apply when the reader is an agent: seven of **EPIC**'s ten dimensions retain characteristics from ISO/IEC/IEEE 29148:2018, and none of the 30 grey-literature sources added a practice. What SDD changes is the unit of quality. The other three dimensions (P1, C1, C2) address a specification that comes with a plan and tasks, and practices such as cross-artifact alignment (I2.3) and requirements-to-implementation traceability (C2.3) concern the relationships among these documents rather than the wording of a single requirement. Participants explained why this matters when an agent does the implementation, describing agents filling gaps with assumptions, overlooking bundled requirements, and declaring incomplete work finished (Section 4.2.1).

Interpreting the Evaluation Results. Every **EPIC** practice received agreement or strong agreement from a majority of participants, but support was not uniform. Normative wording (E1.4), exceptional behavior and technical constraints (P2.2, P2.3), standardized terminology (I2.1), and technical context (C2.1) each received agreement from 14 of 15 participants, which makes them a natural starting point for teams adopting **EPIC**. Participants' reasons for lower ratings on other practices show that a practice's usefulness depends on context. *Timing matters:* P10 questioned detailed delivery planning (P2.4) early in a project, when "*you just want to be able to run it right now*". *Reading cost matters:* P9 doubted that anyone would review agent-generated traceability (C2.3), since "*nobody will be interested in reading all the things that agents created*". *Ownership matters:* participants also differed on where implementation guidance belongs. P2, for example, placed it in agent configuration rather than in a separate plan (C1.2). Teams can therefore adopt **EPIC** incrementally, starting with the broadly endorsed practices and adding others as a project grows and as the cost of an omission rises.

The repository associations in Table 1 are observational, but participants' accounts suggest possible explanations for three of them: lower code complexity, the lower bug-fix rate, and the higher contributor activity. The first explanation concerns scope and constraints. Complexity had the most consistent association, with 15.4% lower median complexity per KLOC in the top band and a decline across the full score range ($\rho = -0.206$). Participants described agents expanding a task beyond what was asked, choosing an unintended technology, or copying the only data model available, and missing constraints forcing the redesign of whole subsystems (Section 4.2.1). Explicit boundaries (E1.5), constraints (P1.2), technical context (C2.1), and data models (C2.4) leave fewer of these decisions to the agent.

The second explanation is verifiability. Participants explained that an agent cannot reliably judge whether its own work is correct or finished: agents may report completion they have not reached, and developers may be unable to interpret the tests an agent writes (Section 4.2.1). Stating measurable outcomes (E3.1), verification methods (E3.2), and completion conditions (E3.4) gives both the agent and the developer a way to check the work without relying on the agent's own report. At least 12 of 15 participants endorsed each of these practices. When a specification states how the work will be checked, defects may be caught before they need later fixes. This explanation is consistent with the lower bug-fix share in top-band repositories (11.8% of scanned commits vs. 20.4% in the bottom band), and the share also declined across the full score range ($\rho = -0.210$).

The third explanation is that the specification may also serve as onboarding documentation. Unclear documentation and poorly documented project structure are known barriers to contribution [73], and repository documentation has been linked to contribution activity [39]. In an SDD project, the specification states the scope, constraints, and completion conditions that both a new contributor and a new agent session need. This role may help explain why top-band repositories had a median of 12.5 contributors compared with 3 in the bottom band, and more than twice as many pull requests (202.5 vs. 89). The relationship may also run the other way: projects with more contributors may invest more in their specifications. These explanations need further testing.

Applying EPIC in Development Workflows. EPIC provides quality criteria that can be applied wherever specification and planning artifacts guide implementation, including workflows without a dedicated SDD framework. SDD frameworks such as OpenSpec and GitHub Spec Kit could use EPIC to guide the clarification questions they ask and the revisions they suggest [32, 41], and teams could check specifications in CI/CD workflows whenever specifications or plans change, building on automated requirements-smell detection [28].

Packaging EPIC as an agent skill. A more portable approach is to give the framework directly to the coding agent. As an initial implementation, we packaged EPIC as an agent skill [4]: a `SKILL.md` instruction file with supporting references. The skill can be reused across projects, which answers Gao et al.'s call to consolidate domain knowledge that each project otherwise writes on its own [36].

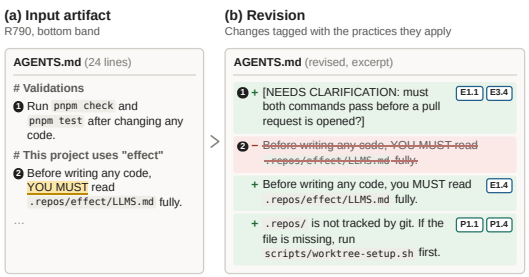

Fig. 3. Applying the EPIC skill: (a) original instructions from R790; (b) instructions revised by the skill, with numbered markers linking problems to changes tagged with EPIC practices.

The skill encodes EPIC's four categories, ten dimensions, and 40 practices, along with an adapted scoring rubric, criteria for what counts as evidence, and examples from the analyzed repositories. In review mode, it examines specification and planning artifacts together, scores each dimension from 0 to 3, and cites passages from the artifacts to support each score. In authoring mode, which covers both drafting new artifacts and revising existing ones, it applies the practices and marks unresolved decisions for clarification. In both modes, it separates practices that do not apply to a feature from information that is missing, and it does not penalize the former.

Figure 3 illustrates the authoring mode. In authoring mode, the skill found that R790's instructions required a file that only a setup step creates, and the instructions never mention that step. The skill added the step, based on the repository, and turned the policies that the file leaves open into clarification questions. These examples show how the skill works; they do not measure how reliably it scores or revises specifications. Because the associations in RQ2 are correlational, a higher score also does not guarantee better code. The skill, its reference files, and the complete outputs for both examples are available in our replication package [8].

How Much Guidance Will Agents Need? Some participants rated practices lower because they expected agents to handle the issue themselves; P3, for example, expected agents to keep artifacts consistent on their own (Section 4.2.1). Agents may handle some omissions well, but whether an agent detects missing information and asks for it varies with the model, prompting, and agent configuration [26, 79]. A specification also has a second reader: the developer who checks the agent's work, who, as participants described, may be unable to judge claims of completion without stated outcomes and verification criteria. Future benchmarks could use EPIC to compare specifications that include, weaken, or omit a practice, such as dependency failure handling (P1.4),

and measure across models and tasks whether the agent asks, decides acceptably, or violates a constraint. Such evidence would show which practices agents can compensate for and which decisions still require the developer.

6 Limitations

Limitations in developing EPIC. Like all empirical studies, our study has limitations. We acknowledge that we may have overlooked quality dimensions or specification practices relevant to SDD when developing **EPIC**. To reduce this risk, we grounded the quality lens in ISO/IEC/IEEE 29148:2018 and SDD tool guidance, derived practices from repository artifacts, and examined practitioner recommendations and interview feedback. These perspectives helped us identify and refine relevant practices.

During the analysis of repository artifacts, scoring and exemplar selection involved researcher judgment. To mitigate potential bias, four researchers first developed the rubric, and we conducted two calibration rounds to assess its consistent application. We complemented the repository findings with grey-literature guidance.

Limitations in evaluating EPIC. The evaluation of **EPIC** provides an initial empirical foundation for SDD practices. However, we did not invite participants to apply **EPIC** in their own projects. Future work will evaluate **EPIC** in use with developers of different backgrounds and experience levels and reassess its coverage and applicability as SDD workflows and agent capabilities evolve.

We chose semi-structured interviews to evaluate **EPIC** because they allowed us to explore participants' explanations, practical constraints, and examples in detail. The 15 participants had varied backgrounds and levels of software development experience, and we screened them for SDD experience to obtain feedback grounded in practice. To reduce participant response bias [21], we presented **EPIC** as a draft, welcomed criticism, and used neutral follow-up questions. The disagreements and explanations reported alongside agreement show that participants were willing to challenge the framework.

The limited repository sample constrains estimate precision and our ability to detect smaller effects. We report effect sizes alongside p-values and apply BH correction [12]. Without multivariate controls or stratified analyses, project characteristics and development practices remain potential confounders, so the associations cannot establish causation. SonarQube results also vary across languages and project structures, while our focus on public, English-language repositories limits generalizability. We treat these findings as exploratory, drawing on complementary statistical tests and practitioner evaluations to examine **EPIC** from multiple perspectives.

7 Conclusion

We introduced **EPIC**, an empirically grounded framework comprising four categories, Expressive, Precise, Interpretable, and Compartmentalized, ten quality dimensions, and 40 practices that guide the writing and review of specification and planning artifacts in SDD. Our future work follows two directions. First, we will evaluate **EPIC** in real development settings with developers of different experience levels, including longitudinal studies in student capstone projects, to understand how the framework affects specification practices and the resulting software. Second, an urgent research priority is to develop benchmarks that examine how coding agents interpret and act on this guidance. Ultimately, as P12 reflected, "*the spec cannot take responsibility for the future; I have to take responsibility for it. To take responsibility for something, I must understand it.*"

Data Availability

The research artifacts for this study are publicly available on the companion website [8], and the **EPIC** framework is available at [7].

References

[1] 2018. ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes – Requirements engineering. *ISO/IEC/IEEE 29148:2018(E)* (2018), 1–104. doi:10.1109/IEEESTD.2018.8559686

[2] Sadia Afroz, Courtney Miller, Tyler Menezes, Edward Gilmour, Anita Sarma, and Zixuan Feng. 2026. " AI Slop is DDoSing Open Source": Understanding the Impact of AI-Generated Contributions on Open Source Sustainability. *arXiv preprint arXiv:2607.04003* (2026).

[3] Shyam Agarwal, Anmol Singhal, Travis Breaux, and Bogdan Vasilescu. 2027. SPECMINE: A Large-Scale Corpus of Spec-Driven Development Artifacts. In *Proceedings of the 24th MSR*. IEEE. arXiv:2608.25202 [cs.SE] doi:10.48550/arXiv.2608.25202

[4] Agent Skills. [n. d.]. Agent Skills Overview. https://agentskills.io.

[5] Adam Alami, Raúl Pardo, and Johan Linåker. 2024. Free open source communities sustainability: Does it make a difference in software quality? *EMSE* 29, 5 (2024), 114.

[6] Idan Amit and Dror G Feitelson. 2021. Corrective commit probability: a measure of the effort invested in bug fixing. *Software Quality Journal* 29, 4 (2021), 817–861.

[7] Anonymous. 2026. EPIC: Specification Quality Framework. https://epicframework.netlify.app/.

[8] Anonymous Anonymous. 2026. *"It's Making a Decision Even If I Don't": The EPIC Framework for Spec-Driven Development — Appendix*. doi:10.5281/zenodo.23114099

[9] Sebastian Baltes and Paul Ralph. 2022. Sampling in software engineering research: A critical review and guidelines. *EMSE* 27, 4 (2022).

[10] Joachim Baumann, Vishakh Padmakumar, Xiang Li, John Yang, Diyi Yang, and Sanmi Koyejo. 2026. Swe-chat: Coding agent interactions from real users in the wild. *arXiv preprint arXiv:2604.20779* (2026).

[11] Kent Beck. 2002. *Test-Driven Development: By Example*. Addison-Wesley Professional, Boston, MA, USA. https://www.informit.com/store/test-driven-development-by-example-9780321146533

[12] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B (Methodological)* 57, 1 (1995), 289–300. doi:10.1111/j.2517-6161.1995.tb02031.x

[13] Leonard Peter Binamungu, Suzanne M. Embury, and Nikolaos Konstantinou. 2018. Maintaining Behaviour Driven Development Specifications: Challenges and Opportunities. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 175–184. doi:10.1109/SANER.2018.8330207

[14] Leonard P. Binamungu, Suzanne M. Embury, and Nikolaos Konstantinou. 2020. Characterising the Quality of Behaviour Driven Development Specifications. In *Agile Processes in Software Engineering and Extreme Programming (Lecture Notes in Business Information Processing, Vol. 383)*. Springer, 87–102. doi:10.1007/978-3-030-49392-9_6

[15] Virginia Braun and Victoria Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. doi:10.1191/1478088706qp063oa

[16] John L Campbell, Charles Quincy, Jordan Osserman, and Ove K Pedersen. 2013. Coding in-depth semistructured interviews: Problems of unitization and intercoder reliability and agreement. *Sociological methods & research* 42, 3 (2013), 294–320.

[17] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E Hassan, et al. 2025. Agent READMEs: An empirical study of context files for agentic coding. *arXiv preprint arXiv:2511.12884* (2025).

[18] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] doi:10.48550/arXiv.2107.03374

[19] Yi-Hung Chou, Boyuan Jiang, Yi Wen Chen, Mingyue Weng, Victoria Jackson, Thomas Zimmermann, and James A. Jones. 2026. Building Software by Rolling the Dice: A Qualitative Study of Vibe Coding. *Proceedings of the ACM on Software Engineering* 3, FSE, Article FSE077 (2026). doi:10.1145/3797105

[20] Jacob Cohen. 1968. Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychological bulletin* 70, 4 (1968), 213.

[21] Nicola Dell, Vidya Vaidyanathan, Indrani Medhi, Edward Cutrell, and William Thies. 2012. " Yours is better!" participant response bias in HCI. In *Proceedings of the sigchi conference on human factors in computing systems*. 1321–1330.

[22] Giuseppe Destefanis, Leila Yousefi, Martin Shepperd, Allan Tucker, Stephen Swift, Steve Counsell, and Mahir Arzoky. 2026. An audit of machine learning experiments on software defect prediction. *Empirical Software Engineering* 31, 4 (2026), 83.

[23] Jessica Díaz, Joaquín Gayoso, Andrea Cimminio, and Jorge Pérez. 2026. Spec-Driven Development for Agentic Software Engineering: Harnessing Human–Agent Teamwork. arXiv:2609.00252 [cs.SE] https://arxiv.org/abs/2609.00252

[24] Berkeley J Dietvorst, Joseph P Simmons, and Cade Massey. 2015. Algorithm aversion: people erroneously avoid algorithms after seeing them err. *Journal of experimental psychology: General* 144, 1 (2015), 114.

[25] Mary T Dzindolet, Scott A Peterson, Regina A Pomranky, Linda G Pierce, and Hall P Beck. 2003. The role of trust in automation reliance. *International journal of human-computer studies* 58, 6 (2003), 697–718.

[26] Nicholas Edwards and Sebastian Schuster. 2026. Ask or assume? uncertainty-aware clarification-seeking in coding agents. *arXiv preprint arXiv:2603.26233* (2026).

[27] Ahmed Fawzy, Amjed Tahir, and Kelly Blincoe. 2026. Vibe Coding in Practice: Motivations, Challenges, and a Future Outlook – a Grey Literature Review. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*. Association for Computing Machinery, 212–223. doi:10.1145/3786583.3786866

[28] Henning Femmer, Daniel Méndez Fernández, Stefan Wagner, and Sebastian Eder. 2017. Rapid quality assurance with requirements smells. *Journal of Systems and Software* 123 (2017), 190–213.

[29] Zixuan Feng, Amreeta Chatterjee, Anita Sarma, and Iftekhar Ahmed. 2022. A case study of implicit mentoring, its prevalence, and impact in Apache. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 797–809.

[30] Zixuan Feng, Reed Milewicz, Emerson Murphy-Hill, Tyler Menezes, Alexander Serebrenik, Igor Steinmacher, and Anita Sarma. 2025. Charting Uncertain Waters: A Socio-Technical Framework for Navigating GenAI's Impact on Open Source Communities. *arXiv e-prints* (2025), arXiv–2508.

[31] Zixuan Feng, Thomas Zimmermann, Lorenzo Pisani, Christopher Gooley, Jeremiah Wander, and Anita Sarma. 2025. When domains collide: An activity theory exploration of cross-disciplinary collaboration. In *2025 ACM/IEEE ESEM*. IEEE, 301–312.

[32] Fission-AI. [n. d.]. OpenSpec: Spec-Driven Development for AI Coding Assistants. Official repository and documentation. Accessed August 31, 2026; initial publication date not verified. https://github.com/Fission-AI/OpenSpec

[33] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. 2021. The SPACE of Developer Productivity: There's more to it than you think. *Queue* 19, 1 (2021), 20–48.

[34] Julian Frattini, Lloyd Montgomery, Jannik Fischbach, Daniel Mendez, Davide Fucci, and Michael Unterkalmsteiner. 2023. Requirements quality research: a harmonized theory, evaluation, and roadmap. *Requirements Engineering* 28, 4 (2023), 507–520.

[35] Mirta Galesic and Michael Bosnjak. 2009. Effects of Questionnaire Length on Participation and Indicators of Response Quality in a Web Survey. *Public Opinion Quarterly* 73, 2 (2009), 349–360. doi:10.1093/poq/nfp031

[36] Haoyu Gao, Jai Lal Lulla, Hong Yi Lin, Sebastian Baltes, Christoph Treude, and Mansooreh Zahedi. 2026. From Registry to Repository: How AI Agent Skills Are Written, Adapted, and Maintained. arXiv:2607.00911 [cs.SE] https://arxiv.org/abs/2607.00911

[37] Vahid Garousi, Michael Felderer, and Mika V. Mäntylä. 2019. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and Software Technology* 106 (2019), 101–121.

[38] D. Randy Garrison, Martha Cleveland-Innes, Marguerite Koole, and James Kappelman. 2006. Revisiting Methodological Issues in Transcript Analysis: Negotiated Coding and Reliability. *The Internet and Higher Education* 9, 1 (2006), 1–8. doi:10.1016/j.iheduc.2005.11.001

[39] Matthew Gaughan, Kaylea Champion, Sohyeon Hwang, and Aaron Shaw. 2025. The introduction of readme and contributing files in open source software development. In *2025 IEEE/ACM 18th International Conference on Cooperative and Human Aspects of Software Engineering (CHASE)*. IEEE, 191–202.

[40] Andrew Gelman and David K Park. 2009. Splitting a predictor at the upper quarter or third and the lower quarter or third. *The American Statistician* 63, 1 (2009), 1–8. doi:10.1198/tast.2009.0001

[41] GitHub. 2025. Spec Kit. Official GitHub repository. Initial commit dated August 21, 2025; accessed August 31, 2026. https://github.com/github/spec-kit

[42] Margaret Hamilton and Saydean Zeldin. 1976. Higher Order Software—A Methodology for Defining Software. *IEEE Transactions on Software Engineering* SE-2, 1 (1976), 9–32. doi:10.1109/TSE.1976.233798

[43] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. 2026. Speed at the cost of quality: How Cursor AI increases short-term velocity and long-term complexity in open-source projects. In *Proceedings of the 23rd international conference on mining software repositories*. 181–193.

[44] L W Heal and Carol K Sigelman. 1995. Response biases in interviews of individuals with limited mental ability. *Journal of intellectual disability research* 39, 4 (1995), 331–340.

[45] Kathryn L. Heninger. 1980. Specifying Software Requirements for Complex Systems: New Techniques and Their Application. *IEEE Transactions on Software Engineering* SE-6, 1 (1980), 2–13. doi:10.1109/TSE.1980.230208

[46] C. A. R. Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (1969), 576–580. doi:10.1145/363235.363259

[47] Brian Houck, Max Kanat-Alexander, Eirini Kalliamvakou, Margaret-Anne Storey, and Nicole Forsgren. 2026. CAFE(S): Your Agent Is Only As Good As Its Context. *Queue* 24, 5 (Sept. 2026). doi:10.1145/3847288

[48] Hsiu-Fang Hsieh and Sarah E Shannon. 2005. Three approaches to qualitative content analysis. *Qualitative health research* 15, 9 (2005), 1277–1288.
[49] Yonghui (Andie) Huang et al. 2026. Regression Accumulation in Multi-Turn LLM Programming Conversations. arXiv:2607.01855 https://arxiv.org/abs/2607.01855
[50] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. 2016. The promises and perils of mining GitHub. *EMSE* 21, 5 (2016), 2035–2071.
[51] Kiro. 2025. Introducing Kiro. Published July 14, 2025; accessed August 31, 2026. https://kiro.dev/blog/introducing-kiro/
[52] Kiro. 2025. Requirements-First. Official documentation. Accessed August 31, 2026. https://kiro.dev/docs/specs/feature-specs/requirements-first/
[53] J. Richard Landis and Gary G. Koch. 1977. The measurement of observer agreement for categorical data. *Biometrics* 33, 1 (1977), 159–174.
[54] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. Association for Computing Machinery, 1–13. doi:10.1145/3597503.3608128
[55] Garm Lucassen, Fabiano Dalpiaz, Jan Martijn E. M. van der Werf, and Sjaak Brinkkemper. 2016. Improving Agile Requirements: The Quality User Story Framework and Tool. *Requirements Engineering* 21, 3 (2016), 383–403. doi:10.1007/s00766-016-0250-x
[56] Daniel Méndez Fernández et al. 2017. Naming the Pain in Requirements Engineering: Contemporary Problems, Causes, and Effects in Practice. *Empirical Software Engineering* 22, 5 (2017), 2298–2338. doi:10.1007/s10664-016-9451-7
[57] Bertrand Meyer. 1992. Applying Design by Contract. *Computer* 25, 10 (1992), 40–51. doi:10.1109/2.161279
[58] Courtney Miller, Sophie Cohen, Daniel Klug, Bogdan Vasilescu, and Christian KaUstner. 2022. " Did you miss my comment or what?" understanding toxicity in open source discussions. In *Proceedings of the 44th international conference on software engineering*. 710–722.
[59] Moran Mizrahi, Guy Kaplan, Dan Malkin, Rotem Dror, Dafna Shahaf, and Gabriel Stanovsky. 2024. State of What Art? A Call for Multi-Prompt LLM Evaluation. *Transactions of the Association for Computational Linguistics* 12 (2024), 933–949. doi:10.1162/tacl_a_00681
[60] Lloyd Montgomery, Davide Fucci, Abir Bouraffa, Lisa Scholz, and Walid Maalej. 2022. Empirical research on requirements quality: a systematic mapping study. *Requirements Engineering* 27, 2 (2022), 183–209.
[61] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, ChenXue Wang, Shichao Liu, and Qing Wang. 2024. ClarifyGPT: A Framework for Enhancing LLM-Based Code Generation via Requirements Clarification. *ACM* 1, FSE, Article 103 (2024), 2332–2354 pages. doi:10.1145/3660810
[62] Nuthan Munaiah, Steven Kroh, Craig Cabrey, and Meiyappan Nagappan. 2017. Curating GitHub for engineered software projects. *EMSE* 22, 6 (2017), 3219–3253.
[63] Robert C. Nickerson, Upkar Varshney, and Jan Muntermann. 2013. A method for taxonomy development and its application in information systems. *European Journal of Information Systems* 22, 3 (2013), 336–359.
[64] Dan North. 2006. Introducing BDD. *Better Software* (March 2006). https://dannorth.net/blog/introducing-bdd/
[65] Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. 2025. An Empirical Study of the Non-Determinism of ChatGPT in Code Generation. *ACM Transactions on Software Engineering and Methodology* 34, 2, Article 42 (2025), 28 pages. doi:10.1145/3697010
[66] Guilherme Paiva, Edna Dias Canedo, and Geraldo Pereira Rocha Filho. 2026. From issue titles to requirements: an empirical study of large language models and prompt engineering strategies. *Requirements Engineering* 31, 1 (2026), 7.
[67] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 754–768.
[68] Deepak Babu Piskala. 2026. Spec-driven development: From code to contract in the age of AI coding assistants. *arXiv preprint arXiv:2602.00180* (2026).
[69] Giovanni Rosa, David Moreno-Lumbreras, Gregorio Robles, and Jesús M González-Barahona. 2026. Understanding specification-driven code generation with LLMs: An empirical study design. *arXiv preprint arXiv:2601.03878* (2026).
[70] Johnny Saldaña. 2021. *Coding techniques for quantitative and mixed data*. Vol. 151. The Routledge reviewer's guide to mixed methods analysis Abingdon.
[71] Helio Victor F dos Santos, Vitor Costa, Joao Eduardo Montandon, Luciana Lourdes Silva, and Marco Tulio Valente. 2026. Configuration Smells in AGENTS. md Files: Common Mistakes in Configuring Coding Agents. *arXiv preprint arXiv:2606.15828* (2026).
[72] Rodrigo Sotolani, Sávio Freire, Felipe Fronchetti, Ronnie de Souza Santos, and Rodrigo Spinola. 2025. Exploring software fairness debt in gray literature. In *Euromicro Conference on Software Engineering and Advanced Applications*. Springer, 85–104.

[73] Igor Steinmacher, Igor Scaliante Wiese, Tayana Conte, Marco Aurélio Gerosa, and David Redmiles. 2014. The hard life of open source software project newcomers. In *Proceedings of the 7th international workshop on cooperative and human aspects of software engineering*. 72–78.

[74] Margaret-Anne Storey, Rashina Hoda, Alessandra Maciel Paz Milani, and Maria Teresa Baldassarre. 2025. Guiding principles for mixed methods research in software engineering. *EMSE* 30, 5 (2025), 138.

[75] Ningzhi Tang, Chaoran Chen, Zihan Fang, Gelei Xu, Maria Dhakal, Yiyu Shi, Collin McMillan, Yu Huang, and Toby Jia-Jun Li. 2026. Programming by Chat: A Large-Scale Behavioral Analysis of 11,579 Real-World AI-Assisted IDE Sessions. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery. doi:10.48550/arXiv.2604.00436

[76] Jonathan Ullrich, Matthias Koch, and Andreas Vogelsang. 2025. From Requirements to Code: Understanding Developer Practices in LLM-Assisted Software Engineering. In *2025 IEEE 33rd International Requirements Engineering Conference (RE)*. IEEE, 257–266. doi:10.1109/RE63999.2025.00032

[77] Muhammad Usman, Ricardo Britto, Jürgen Börstler, and Emilia Mendes. 2017. Taxonomies in software engineering: A systematic mapping study and a revised taxonomy development method. *Information and Software Technology* 85 (2017), 43–59.

[78] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. Association for Computing Machinery, 1–7. doi:10.1145/3491101.3519665

[79] Sanidhya Vijayvargiya, Xuhui Zhou, Akhila Yerukola, Maarten Sap, and Graham Neubig. 2026. Ambig-swe: Interactive agents to overcome underspecificity in software engineering. In *International Conference on Learning Representations*, Vol. 2026. 146716–146737.

[80] Stefan Wagner et al. 2019. Status Quo in Requirements Engineering: A Theory and a Global Family of Surveys. *ACM Transactions on Software Engineering and Methodology* 28, 2 (2019), 1–48. doi:10.1145/3306607

[81] Wenhao Yang, Runzhi He, and Minghui Zhou. 2026. A First Look at Coding Agents' Compliance with AI Contribution Rules in Open-Source Communities. *arXiv preprint arXiv:2607.26819* (2026).

[82] Pamela Zave and Michael Jackson. 1997. Four Dark Corners of Requirements Engineering. *ACM Transactions on Software Engineering and Methodology* 6, 1 (1997), 1–30. doi:10.1145/237432.237434

[83] Yangtian Zi, Harshitha Menon, and Arjun Guha. 2025. More than a score: Probing the impact of prompt specificity on LLM code generation. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*. 2380–2402.

[84] Thomas Zimmermann. 2016. Card-sorting: From text to themes. In *Perspectives on Data Science for Software Engineering*, Tim Menzies, Laurie Williams, and Thomas Zimmermann (Eds.). Morgan Kaufmann, 137–141. doi:10.1016/B978-0-12-804206-9.00027-1

Received 05 October 2026; revised 12 March 2009; accepted 5 June 2009